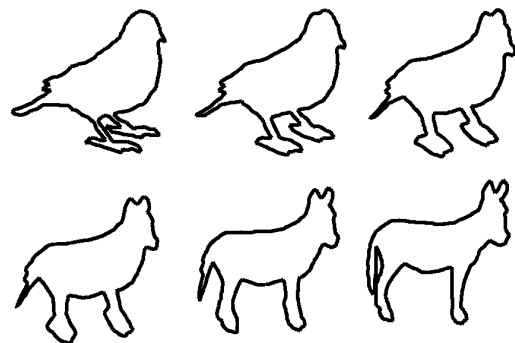




二维多边形形状渐变

2D Shape Blending

蒋浩

Email: jianghao@ict.ac.cn

Shape Blending

- 在计算机动画中，“**Shape blending**”（形状混合）是指在两个或多个不同的**几何形状之间进行插值以产生平滑的过渡**。
- 这种技术经常被用于**角色动画**中，特别是在**角色的姿势变化**和**面部表情动画**中。通过对不同的基础形状（通常被称为“目标形状”或“混合形状”）进行**加权组合**，动画师可以创建一个连续变化的序列，从而使角色的动作和表情看起来更加自然和真实。
- 根据Shape的维度，混合的形状可以是2D、3D、4D、.....
- 是生成**影视特效**的重要方法之一。



After Effects Shapes Blender (2D)

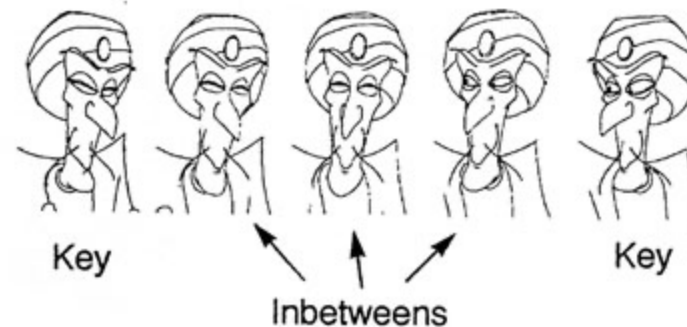


3D Shape Blending



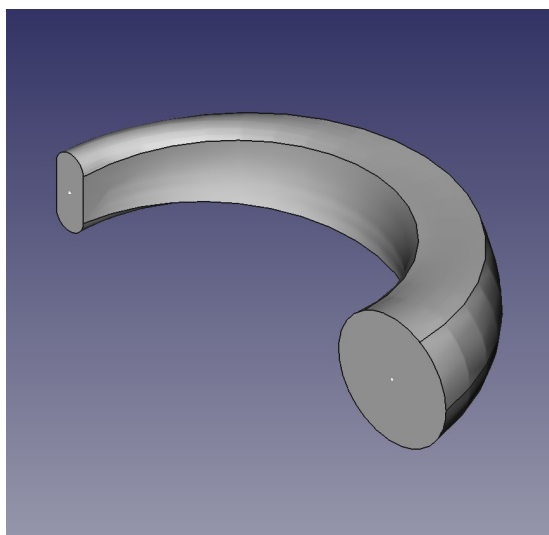
2D Shape Blending问题的引出

- 在二维角色动画(Character Animation)中，经常会碰到这样的问题：给定一个初始和最终的形状(shape)，我们称它们为**关键帧形状**，求从初始形状**光滑过渡**到最终形状的**中间形状**。
- 这个问题称为二维形状的自然渐变 (Shape Blending或Shape Morphing) 。
- 该问题实际上分为两个子问题: (1). **顶点的对应关系问题**; (2). **顶点的插值问题**。

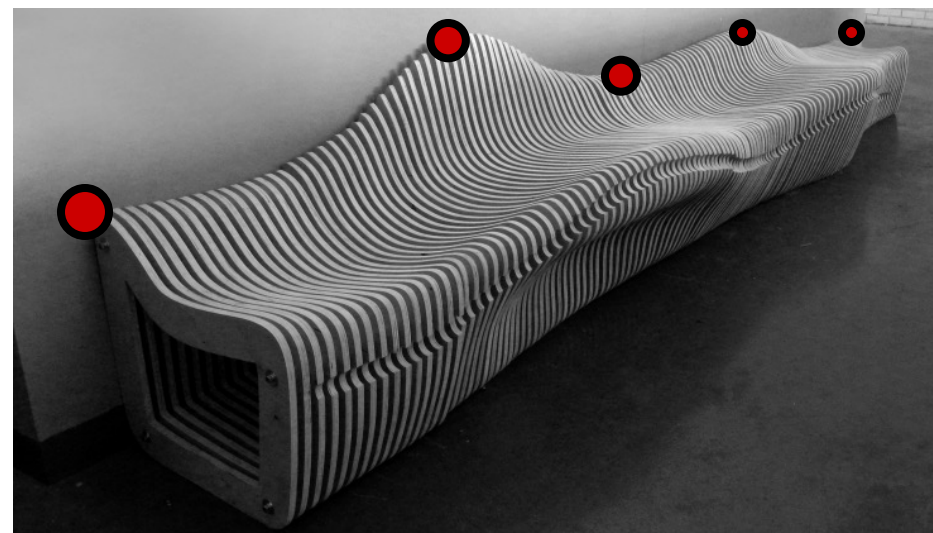
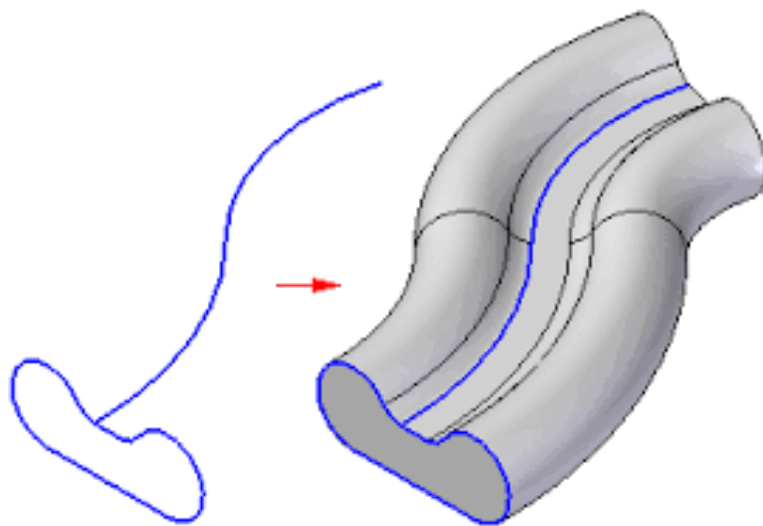


问题的引出

- 二维形状的自然渐变不仅在计算机动画，而且在模式识别、曲面重建和三维造型中也具有重要意义。



Sweeping曲面



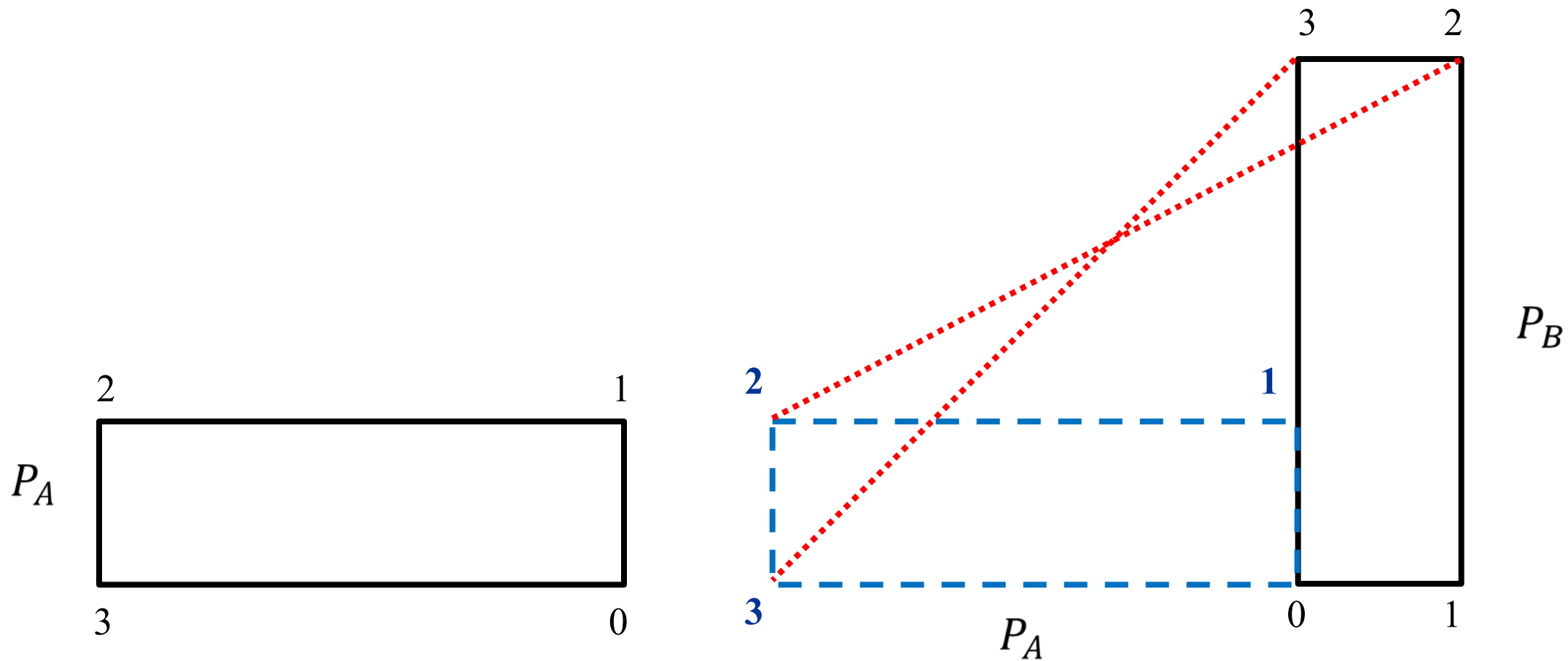
Skin曲面

问题的引出



轮廓

问题的引出



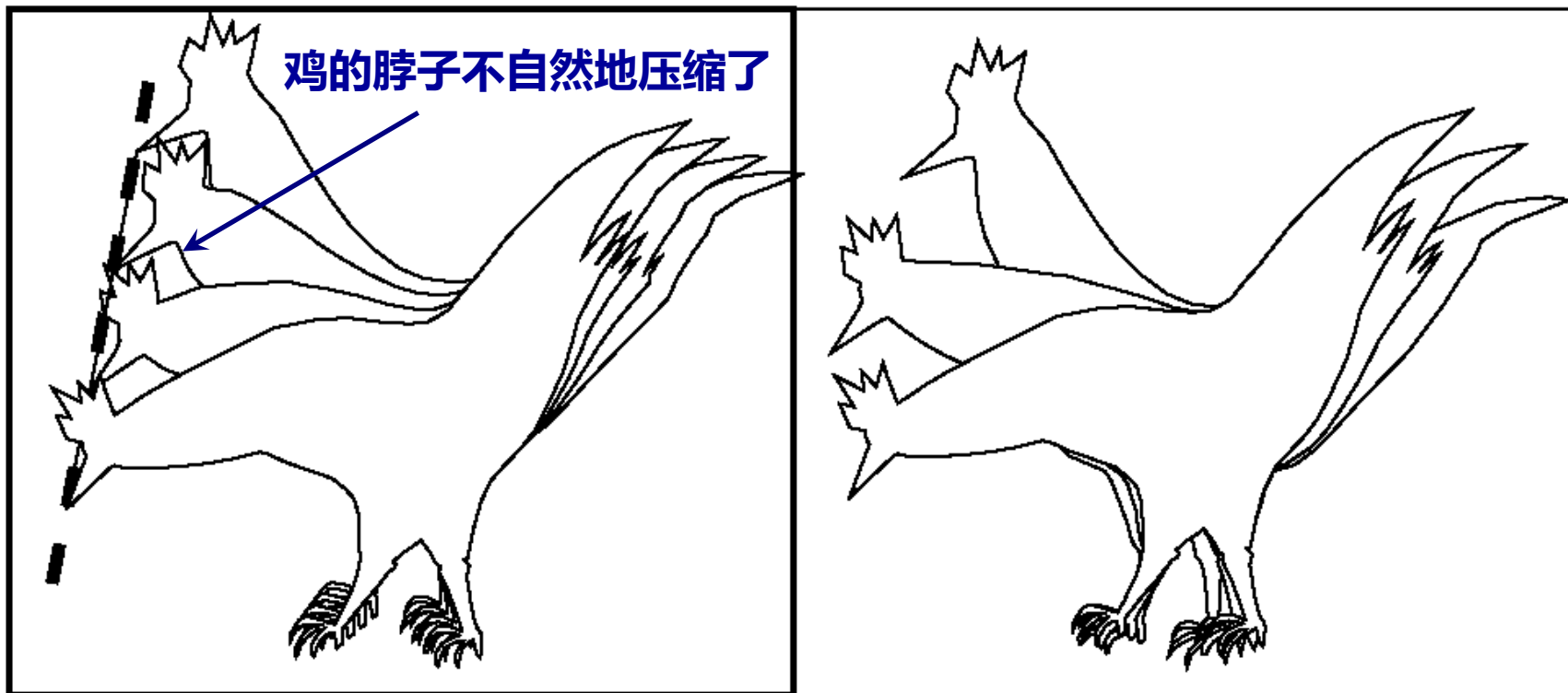
线性插值法

- 假设两个关键帧多边形的顶点为 P_{A_i} 和 P_{B_i} ($i=0, 1, 2, 3, \dots, n$), 顶点的数目都为 n 个, 则我们要解决的是:
 - 顶点的对应关系问题, 即多边形 P_A 中的一个顶点与 P_B 中的哪个顶点对应?
 - 顶点的路径问题, 即 P_A 以何种方式运动到 P_B ?
- 顶点路径问题一个简单的方法为采用**顶点线性插值**:

$$P_i = (1-t)P_{A_i} + tP_{B_i}, \quad (i = 0, 1, 2, \dots, n-1)$$

- 但线性插值会带来**收缩**(shrinkage)和**纽结**现象(Kink), 这在刚体旋转时表现得尤为明显。

线性插值法的缺点



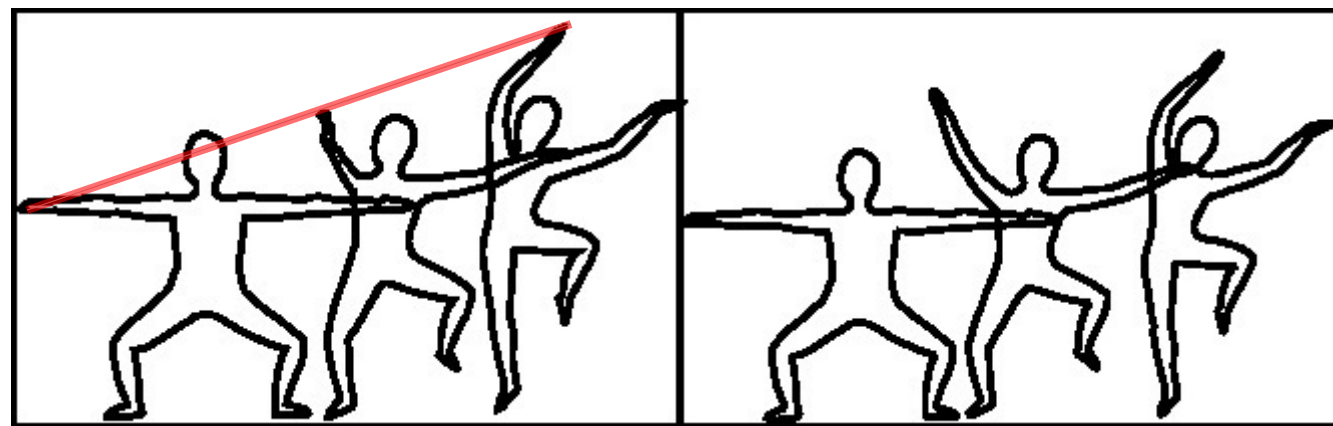
(a) 鸡的脖子压缩了

(b) 更自然的结果

鸡的形状渐变

基于内在形状插值的多边形渐变方法

- Intrinsic Shape Interpolation的数学原理: 乌龟几何+ **Lagrange乘数法优化**



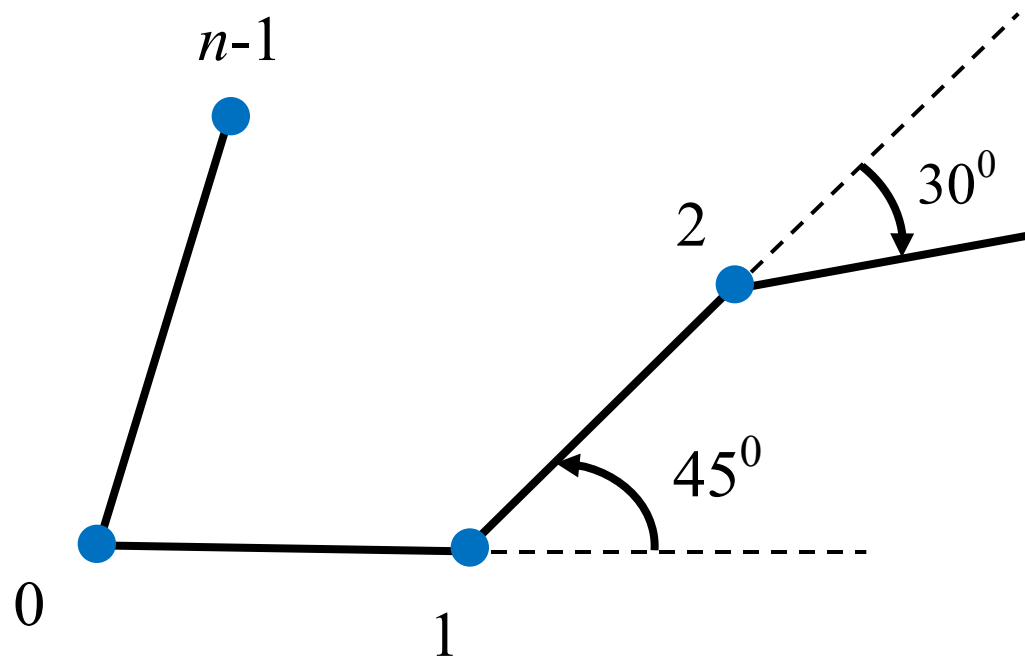
Withered arm

Normal arm

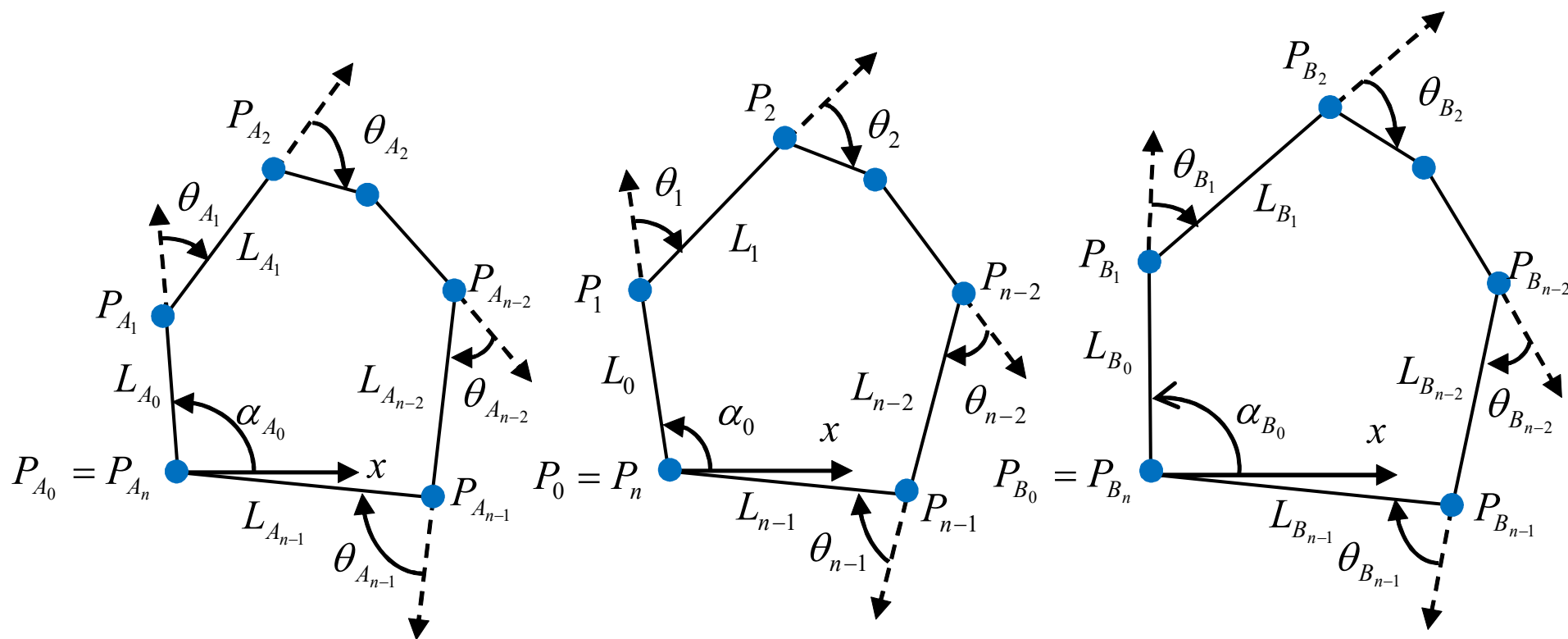
- 参考文献: Sederberg T W, Gao P S, Wang G J, Mu H. 2D shape blending: an intrinsic solution to the vertex path problem. Computer Graphics, 1993, 27(3):15~18

乌龟几何

- 在笛卡尔坐标系中，多边形是通过顶点的坐标显式给出的。
- 但多边形也可以通过乌龟几何来定义，即通过顶点处的边长和有向角来定义。例如，以某一点为起点，向东往前走10米，往左拐 45° ，继续往前走6米，往右拐 30° ，向前走5米，...，最后得到一多边形。
- 因而一个自然的想法是，能否对关键帧多边形的边长和顶点角进行插值来产生比线性插值更好的效果？
- 回答是肯定的，因为这种插值具有更好的几何意义。



多边形的内在定义



- 设 $m=n-1$ ，逆时针方向的角为正的， P_0 为形状的平移锚点(anchor point)，我们的目标是计算中间多边形的顶点 $P_i (i=1, 2, \dots, m)$ ， $0 \leq t \leq 1$ 。
- 通过计算多边形 P_A 和 P_B 的边长和有向顶点角，得到多边形的内在定义 $\{\alpha_0, L_0, (\theta_i, L_i)_{i=1}^m\}$:

$$\begin{aligned} \alpha_{A_0} &= \theta(x, P_{A_0}, P_{A_1}) & \theta_{A_i} &= \theta(P_{A_{i-1}}, P_{A_i}, P_{A_{i+1}}) & L_{A_i} &= |P_{A_{i+1}} - P_{A_i}| \\ \alpha_{B_0} &= \theta(x, P_{B_0}, P_{B_1}) & \theta_{B_i} &= \theta(P_{B_{i-1}}, P_{B_i}, P_{B_{i+1}}) & L_{B_i} &= |P_{B_{i+1}} - P_{B_i}| \end{aligned}$$

其中: $\theta(P_{A_{i-1}}, P_{A_i}, P_{A_{i+1}})$ 表示边 $P_{A_{i-1}}P_{A_i}$ 和边 $P_{A_i}P_{A_{i+1}}$ 之间的有向夹角

中间多边形生成

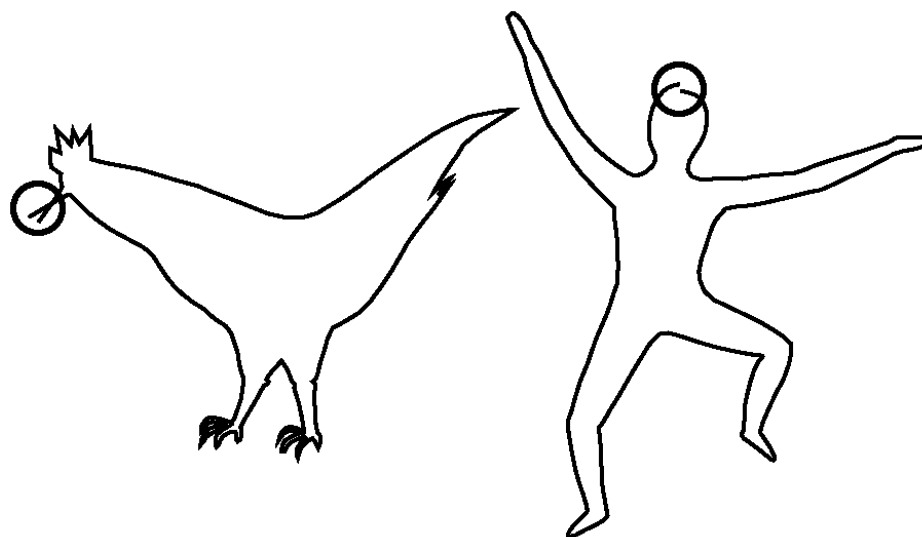
- 那么中间多边形可由插值相应的边长和顶点角得到:

$$\alpha_0 = (1-t)\alpha_{A_0} + t\alpha_{B_0}$$

$$\theta_i = (1-t)\theta_{A_i} + t\theta_{B_i} \quad (i = 1, 2, \dots, m)$$

$$L_i = (1-t)L_{A_i} + tL_{B_i} \quad (i = 1, 2, \dots, m)$$

- 遗憾的是，这样得到的多边形通常是不封闭的。但实验发现，得到多边形的起始点和终止点非常接近。



不封闭的多边形

中间多边形生成

- 一个解决方法为保持插值的顶点角不变，适当调整插值得到的边长(Edge Tweaking):

$$L_i = (1-t)L_{A_i} + tL_{B_i} + S_i, \quad (i = 0, 1, 2, \dots, m)$$

- $|S_i| \propto |L_{A_i} - L_{B_i}|$ ，如果关键多边形的某一条对应的边具有相同的长度 L ，那么我们可以认为中间多边形的边长也为 L 。为了防止除以零，我们定义：

$$L_{AB_i} = \max\left\{|L_{A_i} - L_{B_i}|, L_{tol}\right\}, \quad (i = 0, 1, 2, \dots, m)$$

其中： $L_{tol} = 0.0001 \times (\max_{i \in [0, m]} |L_{A_i} - L_{B_i}|)$

中间多边形生成

- 我们的目标是为了求得 $S_0, S_1, \dots, S_m$, 使目标函数:

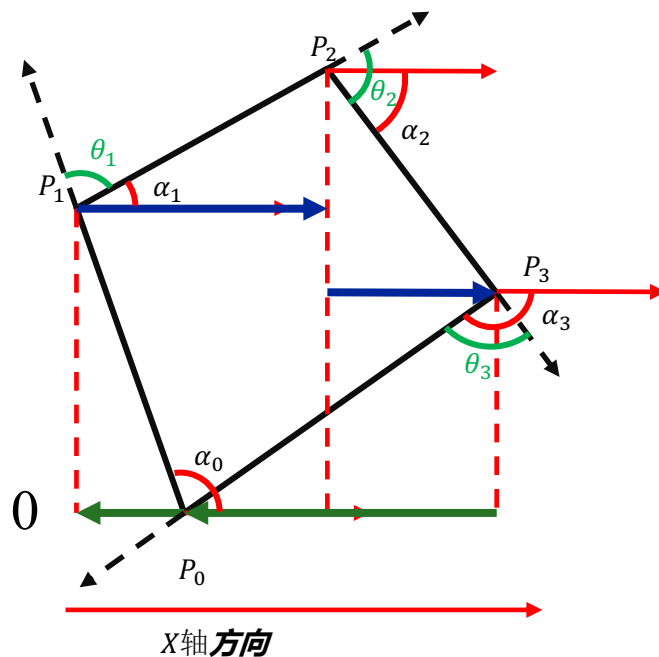
$$f(S_0, S_1, \dots, S_m) = \sum_{i=0}^m \frac{S_i^2}{L_{AB_i}^2}$$

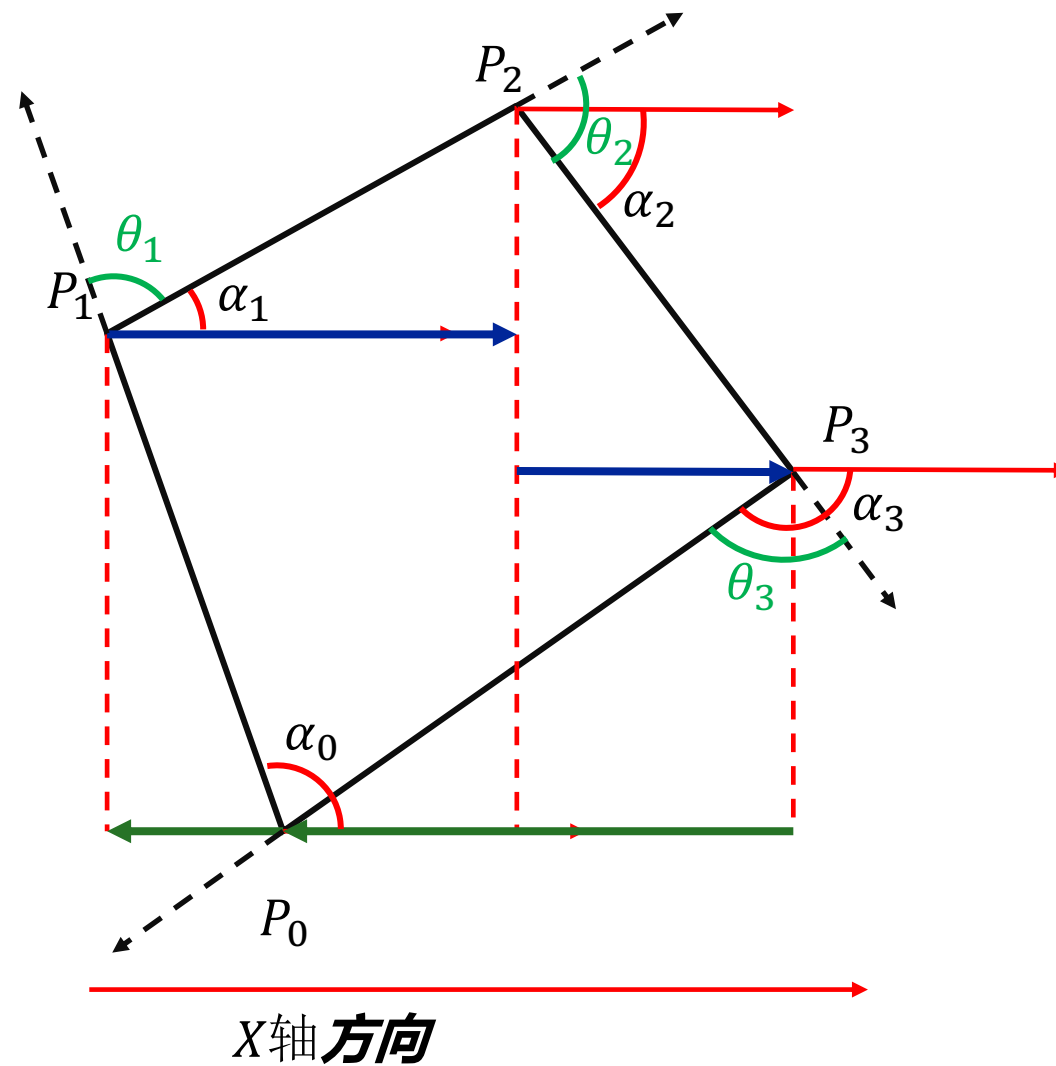
最小, 并且 $S_0, S_1, \dots, S_m$ 应满足约束条件(强迫多边形封闭):

$$\varphi_1(S_0, S_1, \dots, S_m) = \sum_{i=0}^m [(1-t)L_{A_i} + tL_{B_i} + S_i] \cos \alpha_i = 0$$

$$\varphi_2(S_0, S_1, \dots, S_m) = \sum_{i=0}^m [(1-t)L_{A_i} + tL_{B_i} + S_i] \sin \alpha_i = 0$$

其中 α_i 是由矢量 $P_i P_{i+1}$ 和 x 轴构成的有向角: $\alpha_i = \alpha_{i-1} + \theta_i, (i = 1, 2, \dots, m)$





中间多边形生成

- 这是个具有约束条件的极值问题，可用Lagrange乘数法来求解。引进Lagrange函数：

$$\Phi(\lambda_1, \lambda_2, S_1, S_2, \dots, S_m) = f + \lambda_1 \varphi_1 + \lambda_2 \varphi_2$$

其中 λ_1, λ_2 为Lagrange乘数。对 Φ 求偏导得：

$$\frac{\partial \Phi}{\partial S_i} = \frac{2S_i}{L_{AB_i}^2} + \lambda_1 \cos \alpha_i + \lambda_2 \sin \alpha_i = 0 \quad (i = 0, 1, \dots, m)$$

$$\sum_{i=0}^m [(1-t)L_{A_i} + tL_{B_i} + S_i] \cos \alpha_i = 0$$

$$\sum_{i=0}^m [(1-t)L_{A_i} + tL_{B_i} + S_i] \sin \alpha_i = 0$$

中间多边形生成

- 由第一式求得 $S_i = -\frac{L_{AB}^2}{2} (\lambda_1 \cos \alpha_i + \lambda_2 \sin \alpha_i)$

- 代入后二式得:

$$\begin{cases} E\lambda_1 + F\lambda_2 = U \\ F\lambda_1 + G\lambda_2 = V \end{cases}$$

其中:

$$E = \sum_{i=0}^m L_{AB_i}^2 \cos^2 \alpha_i, F = \sum_{i=0}^m L_{AB_i}^2 \sin \alpha_i \cos \alpha_i, G = \sum_{i=0}^m L_{AB_i}^2 \sin^2 \alpha_i$$

$$U = 2 \left\{ \sum_{i=0}^m \left[(1-t)L_{A_i} + tL_{B_i} \right] \cos \alpha_i \right\}$$

$$V = 2 \left\{ \sum_{i=0}^m \left[(1-t)L_{A_i} + tL_{B_i} \right] \sin \alpha_i \right\}$$

中间多边形生成

- 如果 $EG - F^2 \neq 0$, 则

$$\lambda_1 = \left| \begin{array}{cc} U & F \\ V & G \end{array} \right| / \left| \begin{array}{cc} E & F \\ F & G \end{array} \right|$$

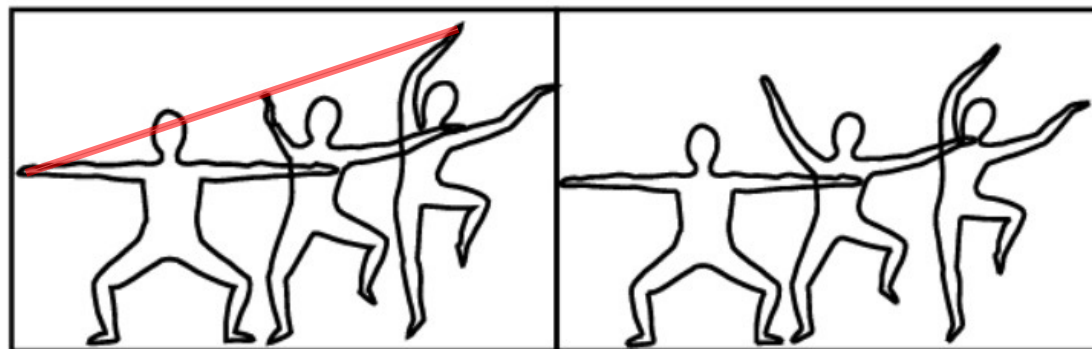
$$\lambda_2 = \left| \begin{array}{cc} E & U \\ F & V \end{array} \right| / \left| \begin{array}{cc} E & F \\ F & G \end{array} \right|$$

$$S_i = -\frac{1}{2} L_{AB_i}^2 (\lambda_1 \cos \alpha_i + \lambda_2 \sin \alpha_i), (i = 0, 1, 2, \dots, m)$$

从而可以计算得到 L_i , 从而可得到中间多边形顶点 (x_i, y_i) 的坐标:

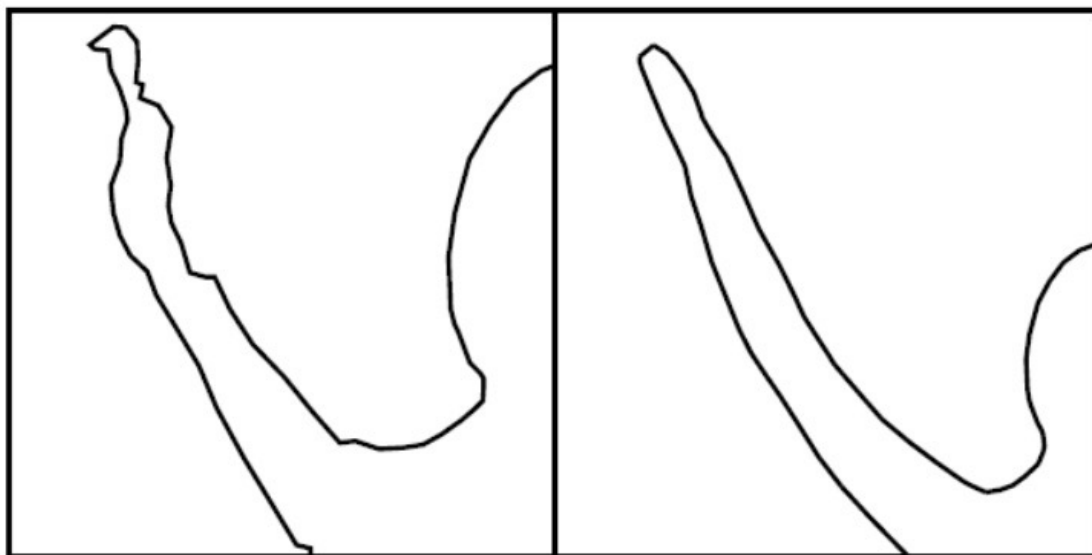
$$\begin{cases} x_i = x_{i-1} + L_{i-1} \cos \alpha_{i-1} \\ y_i = y_{i-1} + L_{i-1} \sin \alpha_{i-1} \end{cases}$$

与线性插值的结果对比



a. Withered Arm

b. Normal Arm

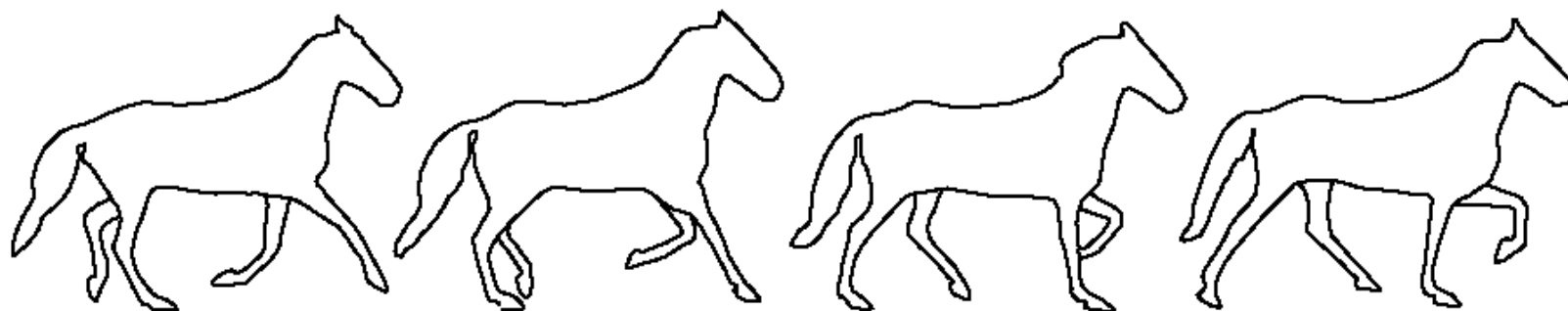


Linear vertex paths

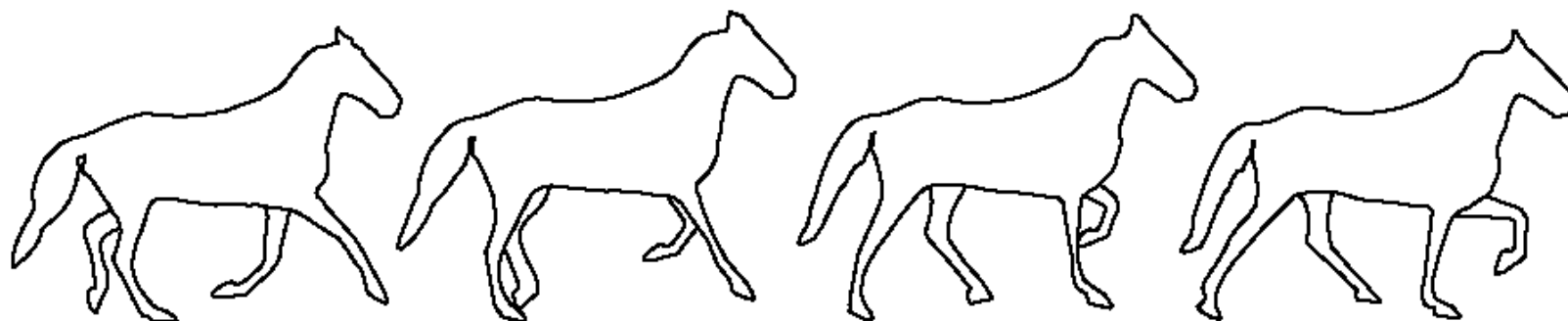
Intrinsic blend

结果

- 实验表明，对于角色动画中的许多应用，内在插值的形状渐变效果良好。



数字化得到的形状(形状取自经典摄影图书《Animals in Motion》)



给定的

内在插值得到的

内在插值得到的

给定的

算法实现

输入： 两个关键帧多边形的顶点

- 计算两个关键帧多边形的内在表示;
- 解线性方程组, 计算 λ_1, λ_2 ;
- 计算 S_i ;
- 中间多边形顶点的坐标;

输出： 中间帧多边形的顶点

DEMO

*MORPHING Based on
Strain Field Interpolation*

gymnastic patterns

CASA 2004

内在形状插值法的优缺点

- **优点：**

- 简单；
- 能在一定程度上避免形状插值中出现的收缩和纽结现象，这在二维角色动画中尤其有用。

- **缺点：**

- 顶点对应关系需要用户交互给出；
- 结果依赖于顶点对应关系的好坏；
- 当源和目标多边形中包含短边时，由于计算短边的方向不稳定，中间帧多边形有可能产生较严重的畸变。

基于模糊数学的二维多边形形状渐变

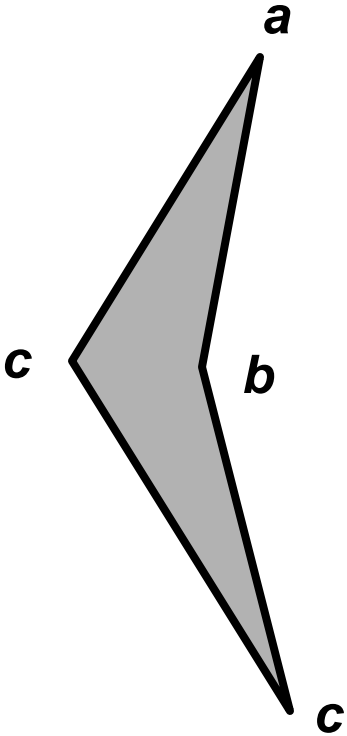
- 1996年, Zhang提出了一种基于模糊数学的二维多边形形状渐变方法。
- 它采用基于模糊数学的方法来建立源多边形和目标多边形**顶点的对应关系**以及**顶点之间的插值**。
- **优点:**
 - 能处理具有不同位置、方向、大小或形状的多边形。该方法不仅稳定, 而且易于推广到曲线形状。

* Zhang Yuefeng. A fuzzy approach to digital image warping. IEEE Computer Graphics and Applications, 1996, 16(6):34~41

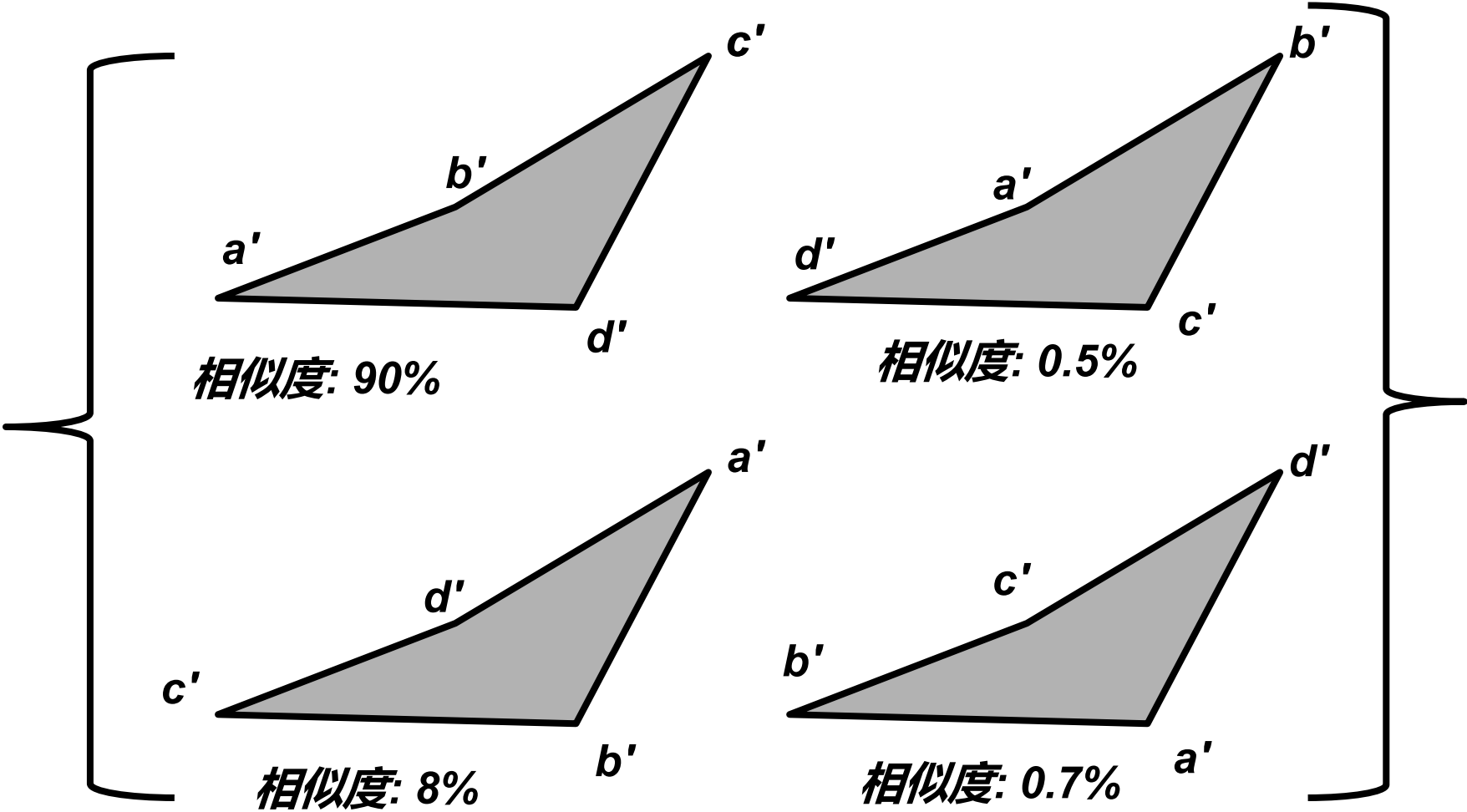
基于模糊数学的顶点对应关系的建立

- 顶点对应关系在多边形形状渐变中占据非常重要的地位。
 - 一个好的对应关系可以指导插值的过程，并生成光滑流畅的中间帧；
 - 而一个不好的对应关系则会生成不自然的中间帧。
- 如果源多边形和目标多边形具有不同的位置、方向、大小或形状，在没有人工干预的情况下，要找到一个好的对应关系通常是件困难的事。当两个多边形的顶点个数不同时，问题就更加复杂。
- **思想：**基于模糊数学，用目标多边形建立一个与给定源多边形相似的模糊多边形集，然后在模糊集中查找多边形相似函数最大的多边形。

模糊多边形集示例



源多边形



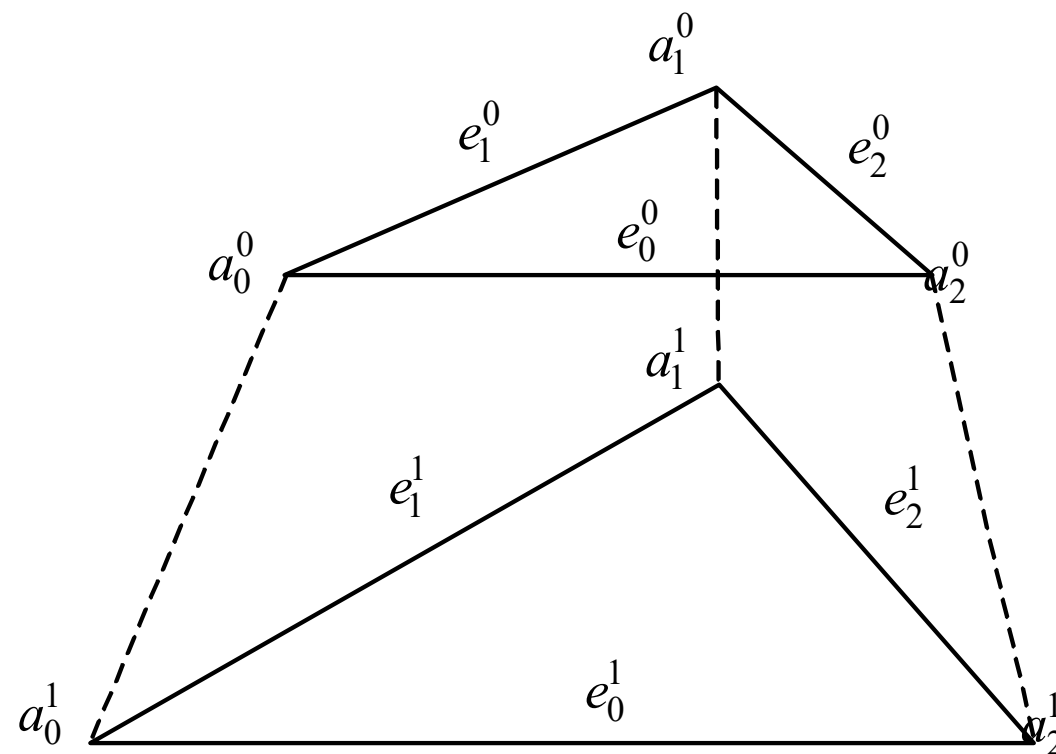
目标多边形

多边形相似函数

- 对于两个多边形 $P^0 = \{a_0^0, a_1^0, \dots, a_n^0\}$ 和 $P^1 = \{a_0^1, a_1^1, \dots, a_n^1\}$, 如果它们对应的角相等且对应的边成比例, 则称它们是相似的。
- 两个多边形的相似程度由多边形相似函数来决定。
 - 若多边形相似函数的值很高, 则这两个多边形的相似程度很高;
 - 若多边形相似函数的值很低, 则这两个多边形的相似程度很低。

多边形相似函数

- **多边形相似函数**：对应角相同的程度和对应边成比例的程度。
- 对于**两个多边形**，注意到如果由对应的角和形成该角的两条边组成的三角形相似，则这两个多边形相似。因此我们可以用这些三角形来定义相似函数。



两个相似三角形

多边形相似函数: 三角形的相似函数

- 若这两个三角形相似, 则 $e_1^0 \times e_2^1 = e_1^1 \times e_2^0$, 且 $a_1^0 = a_1^1$
- 若它们不完全相似, 则 $e_1^0 \times e_2^1 \neq e_1^1 \times e_2^0$, 或 $a_1^0 \neq a_1^1$
- 多边形的相似程度依赖于这种不同的程度。因此我们可根据这种不同来定义**三角形的相似函数**:

$$\text{sim}_t = w_1 \times \left(1 - \frac{|e_1^0 \times e_2^1 - e_1^1 \times e_2^0|}{e_1^0 \times e_2^1 + e_1^1 \times e_2^0}\right) + w_2 \times \left(1 - \frac{|a_1^0 - a_1^1|}{360}\right)$$

其中 $0 < w_1, w_2 < 1$, 且 $w_1 + w_2 = 1$ 。可设 $w_1 = w_2 = 0.5$

该相似函数具有以下性质:

- $0 \leq \text{sim}_t \leq 1$
- 如果两个三角形相似, 则 $\text{sim}_t = 1$;
- 如果两个三角形较相似, 则 sim_t 较大;
- 如果两个三角形有很大不同, 则 sim_t 较小;

多边形相似函数

- 给定一顶点对应关系映射 $P^0 \rightarrow P^1$, 我们可以利用三角形的相似函数来定义多边形 P^0 和 P^1 的相似函数:

$$\text{sim}_p(P^0, P^1) = \frac{1}{n+1} \sum_{i=0}^n \text{sim}_t(T_i^0, T_j^1)$$

其中 $j=\text{map}(i)$, T_i^0 和 T_j^1 为由多边形对应角 a_i^0 和 a_j^1 以及它们相邻的边 e_i^0, e_{i+1}^0 和 e_j^1, e_{j+1}^1 组成的三角形, $\text{sim}_t(T_i^0, T_j^1)$ 为相应三角形的相似函数。

- 为了方便起见, 我们把由多边形的角和其相邻边组成的三角形称为**多边形角**。

函数 Sim_p 具有以下性质:

- $0 \leq \text{sim}_p \leq 1$;
- 如果两个多边形相似, 则 $\text{Sim}_p = 1$;
- 如果两个多边形较相似, 则 Sim_p 较大;
- 如果两个多边形有很大不同, 则 Sim_p 较小;

模糊集

- 若把一个多边形看成一系列有序的多边形顶点，则对于相同的多边形，取不同的顶点为起始点，我们可以得到多个多边形。不妨设多边形的顶点按顺时针方向排列。
- 对于 n 个顶点的多边形，可得到 n 个多边形。例如，对于目标多边形 P^1 ，我们可以得到 $n+1$ 个多边形 $P^{1,i}, i = 0, 1, \dots, n$ ，其中：

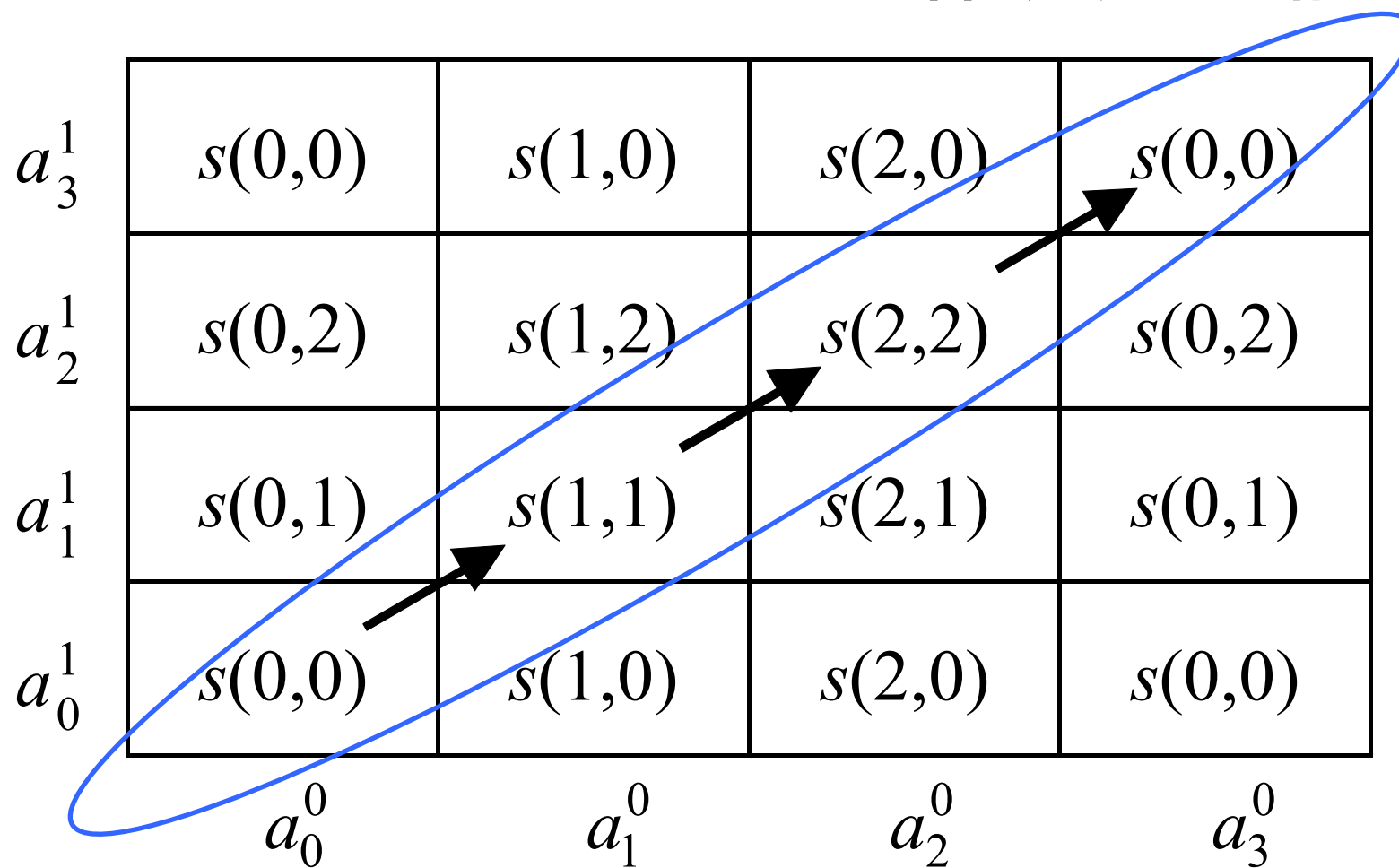
$$P^{1,i} = \{a_i^1, a_{(i+1) \bmod (n+1)}^1, \dots, a_{(i+n) \bmod (n+1)}^1\}$$

- 给定源多边形 P^0 ，多边形集 $\{P^{1,i}\}$ 和多边形相似函数 Sim_p 构成了模糊集。属于该集合的多边形 X 和 P^0 的相似程度为 $Sim_p(P^0, X)$ 。

图论求解方法

- 应用模糊数学技术，建立多边形 P^0 和 P^1 的对应关系问题转化成为在模糊集中寻找多边形 P ，使得相应的**隶属函数值最大**。
- Zhang采用了一种基于图论的方法来求解。
- 给定多边形 $P^0 = \{a_0^0, a_1^0, \dots, a_m^0\}$ 和 $P^1 = \{a_0^1, a_1^1, \dots, a_n^1\}$, $m \geq n$; 每对对应多边形角的模糊相似度可由节点数为 $(m+2) \times (n+2)$ 的图 $G[(m+2), (n+2)]$ 来表示, 其中 $G[i, j]$ 表示多边形角 a_i^0 和 a_j^1 的模糊相似程度, $0 \leq i \leq m+1, 0 \leq j \leq n+1$,
 $G[i, n+1] = G[i, 0]$, $G[m+1, j] = G[0, j]$
- 我们称这样的图为**模糊相似图**。

合法又完全的路径



多边形的模糊相似图, $s(i, j) = \text{sim}_t(a_i^0, a_j^1)$

路径

a_3^1	$s(0,0)$	$s(1,0)$	$s(2,0)$	$s(0,0)$
a_2^1	$s(0,2)$	$s(1,2)$	$s(2,2)$	$s(0,2)$
a_1^1	$s(0,1)$	$s(1,1)$	$s(2,1)$	$s(0,1)$
a_0^1	$s(0,0)$	$s(1,0)$	$s(2,0)$	$s(0,0)$
	a_0^0	a_1^0	a_2^0	a_3^0

- 在相似图中，一条路径的代价(cost)为该路径中每个节点相似度的和。
- Zhang发现，如果 P^0 中的每个顶点只对应于 P^1 中的一个顶点时，得到的结果较好（假设 P^0 的顶点数 $m \geq n$ ）。
- 这意味着在相似图中，不允许有向“北”、“南”的跨步，只允许“东”或“东北”的跨步。这样得到的路径称为是一条合法的路径。如果路径上的节点表示了多边形的顶点对应关系，则称该路径是完全的路径。
- 因此，使相似函数最大的两多边形顶点对应关系问题转化为在模糊相似图中寻找一条最长的合法、完全的路径问题。

合法但不完全的路径

a_3^1	$s(0,0)$	$s(1,0)$	$s(2,0)$	$s(0,0)$
a_2^1	$s(0,2)$	$s(1,2)$	$s(2,2)$	$s(0,2)$
a_1^1	$s(0,1)$	$s(1,1)$	$s(2,1)$	$s(0,1)$
a_0^1	$s(0,0) \rightarrow s(1,0)$	$s(2,0)$	$s(0,0)$	
	a_0^0	a_1^0	a_2^0	a_3^0

多边形的模糊相似图

图论求解方法

- 为了利用标准的图论算法，我们考虑图 G 的余图 G^{-1} ，其中图 G^{-1} 中的元素为 $G^{-1}[i,j]=1-G[i,j]$;

a_3^1	$l-s(0,0)$	$l-s(1,0)$	$l-s(2,0)$	$l-s(0,0)$
a_2^1	$l-s(0,2)$	$l-s(1,2)$	$l-s(2,2)$	$l-s(0,2)$
a_1^1	$l-s(0,1)$	$l-s(1,1)$	$l-s(2,1)$	$l-s(0,1)$
a_0^1	$l-s(0,0)$	$l-s(1,0)$	$l-s(2,0)$	$l-s(0,0)$
	a_0^0	a_1^0	a_2^0	a_3^0

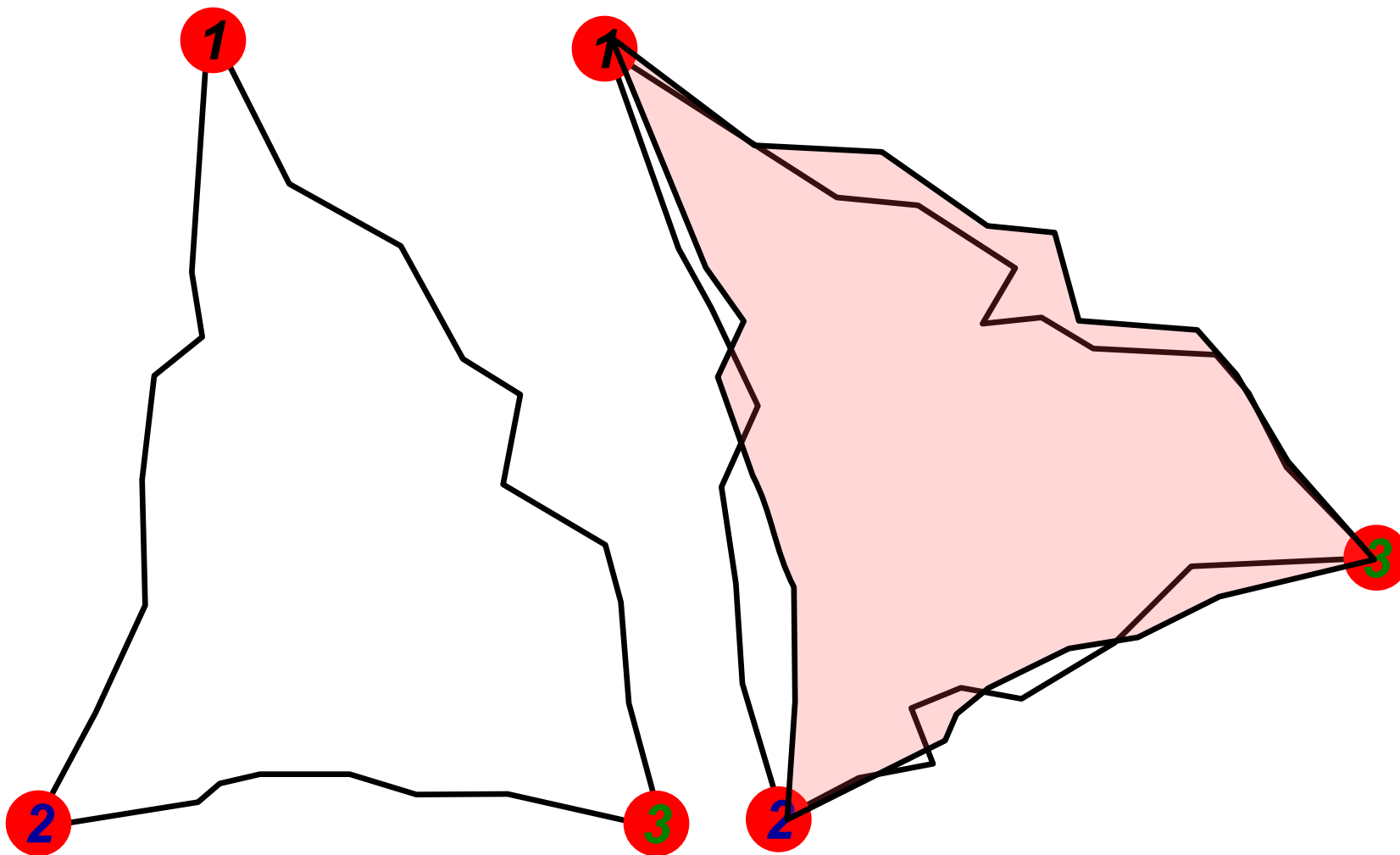
- 显然，在图 G 中寻找最长合法路径问题变成了在图 G^{-1} 中寻找最短的合法路径问题。
- 给定图 G^{-1} 中初始节点 $[i_1, j_1]$ 和终止节点 $[i_2, j_2]$ ，我们可用标准的图论算法来决定连接这两个节点的最短合法路径。

多边形的对应关系由其中最短的一条来建立!

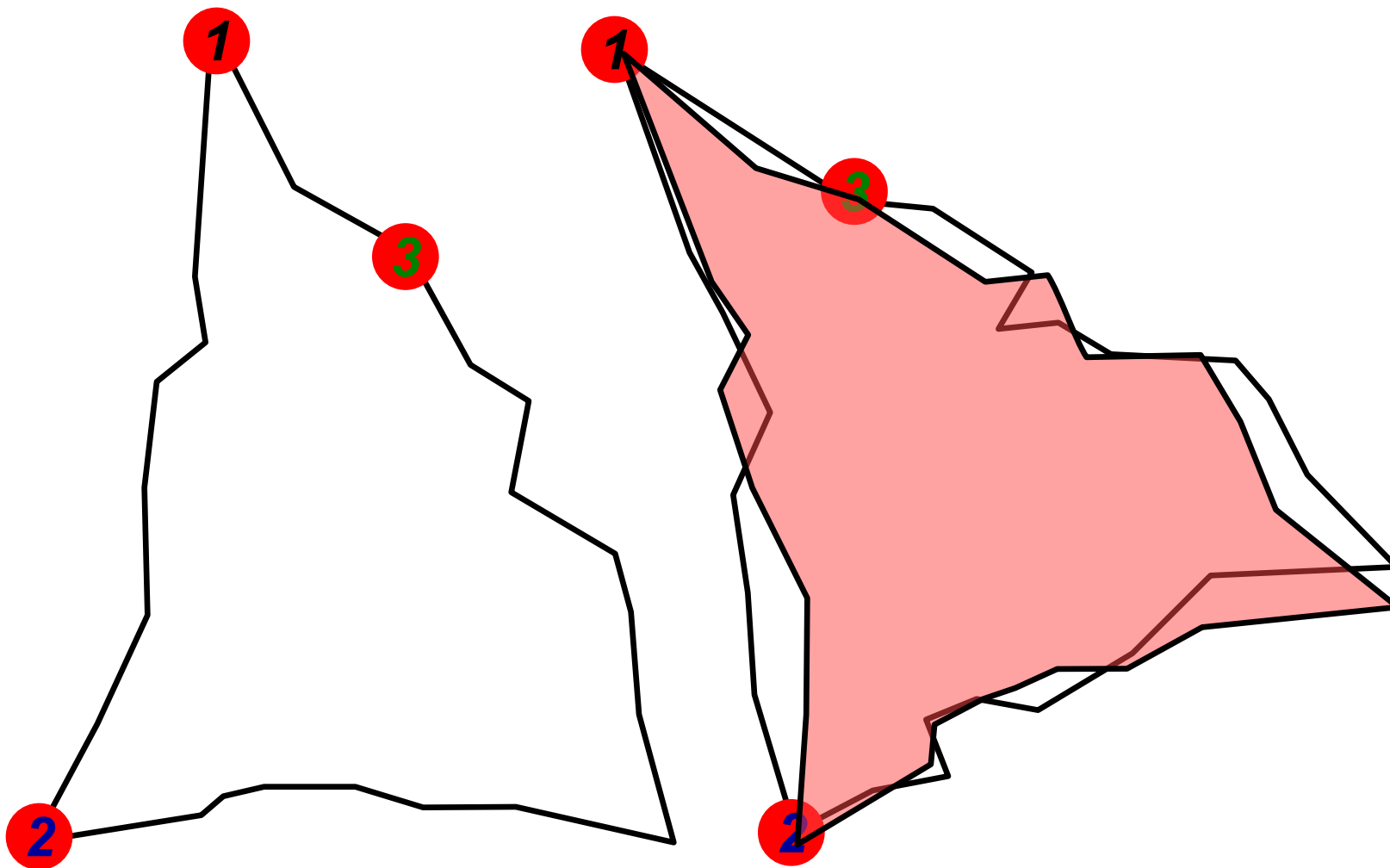
仿射变换的选取

- 为了插值，需要求一个把源多边形变换到目标多边形的好的仿射变换。
- 我们知道三对顶点决定了一个唯一的仿射变换。给定两个多边形，所有三对对应多边形顶点决定了一个仿射变换集。
- 为了选择一个好的仿射变换，考虑由上述仿射变换构成的模糊仿射变换集 $T = \langle y, \text{smooth}_t \rangle$ ，其中仿射变换函数 smooth_t 表示把源多边形光滑过渡到目标多边形的“好坏”程度。
- 我们的策略: 基于一对对应多边形角的好坏函数 smooth_a 来决定 smooth_t 。

仿射变换的选取



仿射变换的选取



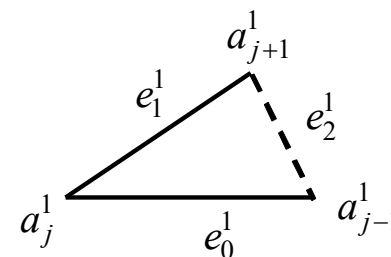
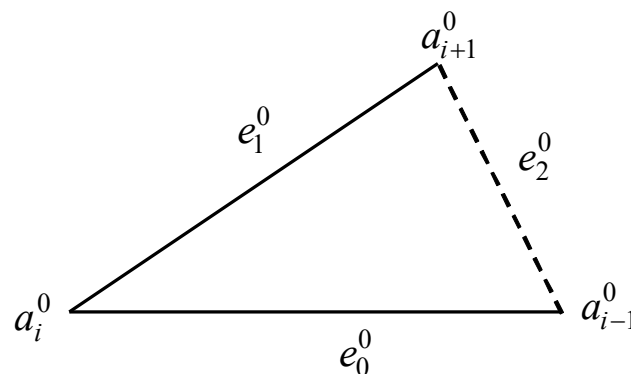
仿射变换的选取

- 给定一对对应角 a_i^0 和 a_j^1 ，我们考虑对应三角形的如下因素来决定 smooth_a ：

1. 三角形 $T_i^0 = a_{i-1}^0 a_i^0 a_{i+1}^0$ 和 $T_j^1 = a_{j-1}^1 a_j^1 a_{j+1}^1$ 在形状和大小上的相似程度 S ；

$$S = w_1 \times \left(1 - \frac{\sum_{i=0}^2 |e_i^0 - e_i^1|}{\sum_{i=0}^2 |e_i^0 + e_i^1|}\right) + w_2 \times \left(1 - \frac{\sum_{i=0}^2 |a_i^0 - a_i^1|}{180}\right)$$

$0 < w_1, w_2 < 1, w_1 + w_2 = 1$, 执橱□□□废□罍0.5.



一对对应的多边形角

2. 把 T_i^0 变换为 T_j^1 所需的最小旋转角 R 。它可由三角形的顶点来求得。
3. 两个三角形的面积之和与两个多边形面积之和之比 A 。

$$\text{smooth}_a = a \times S + b \times \left(1 - \frac{R}{180}\right) + c \times A$$

仿射变换的选取

- 给定三对对应角 (a_i^0, a_i^1) , (a_j^0, a_j^1) 和 (a_k^0, a_k^1) , 定义对应仿射变换的 “好坏” 函数为:

$$\text{smooth}_t(T(i, j, k)) = \text{smooth}_a(a_i^0, a_i^1) \times \text{smooth}_a(a_j^0, a_j^1) \times \text{smooth}_a(a_k^0, a_k^1)$$

- 在模糊仿射变换集中 “好坏” 函数最大的仿射变换, 即为我们所选取的好的仿射变换 (三对对应顶点) 。
- 注意, 这样决定的三对对应顶点**并不需要是相邻的**。

仿射变换的插值

- 现在讨论仿射变换的插值问题。采用齐次矩阵表示，一个仿射变换可写成如下形式：

$$(u', v') = (u, v) \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} + (a_{31}, a_{32}) = (u, v) \mathbf{A} + \mathbf{T}$$

在几何变换中， $\mathbf{T}$ 为平移向量，而 $\mathbf{A}$ 为表示旋转、比例缩放和剪切变换的复合变换。

- 插值仿射变换的一个方法是线性插值复合旋转矩阵和平移向量，即

$$(1-t) \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + t \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

和 $t(a_{31}, a_{32})$ ，其中 $0 \leq t \leq 1$ 。

仿射变换的插值

- 但线性插值并不能得到光滑的变换。为了得到更流畅的结果，我们把复合矩阵 **A** 分解成如下形式：

$$\mathbf{A} = \mathbf{BC} = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix} \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

其中 **B** 为一纯粹的旋转矩阵。

- 给定矩阵 **A**，矩阵 **B** 和 **C** 的计算方法如下：

$$\mathbf{B} = \mathbf{A} + \text{sign}(\det(\mathbf{A})) \begin{pmatrix} a_{22} & -a_{21} \\ -a_{12} & a_{11} \end{pmatrix}$$

$$\mathbf{C} = \mathbf{B}^{-1} \mathbf{A}$$

仿射变换的插值

- 由于B为一**纯粹的旋转矩阵**，可把它写成

$$\begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

的形式。从而我们得到一更流畅的复合矩阵插值公式：

$$(1-t)\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \begin{pmatrix} \cos(t\theta) & -\sin(t\theta) \\ \sin(t\theta) & \cos(t\theta) \end{pmatrix} \begin{pmatrix} tc_{11} & tc_{12} \\ tc_{21} & tc_{22} \end{pmatrix}$$

- 该方法使**显式插值**多边形的**方向**成为可能。

计算中间帧多边形

- 现在我们讨论怎样把中间帧的仿射变换应用于源多边形 P^0 ，从而得到中间帧的多边形。
 - 在仿射变换的选取中，我们得到了源多边形和目标多边形的三对对应顶点： $(A_1, A_2), (B_1, B_2), (C_1, C_2)$ ，它们构成了这两个多边形的局部坐标系，根据前面所述的方法插值得到中间帧的仿射变换，并把它作用于顶点 A_1 ， B_1 和 C_1 来生成三个中间帧顶点 A ， B 和 C
 - 对多边形的所有顶点对，计算顶点 X_1 和 X_2 的插值点 X 在中间帧局部坐标系中的坐标

计算中间帧多边形

- 设 X_1, X_2 为源多边形和目标多边形上的点，它们在相应局部坐标系上的规范化局部坐标分别为 (u_1, v_1) 和 (u_2, v_2) ，则

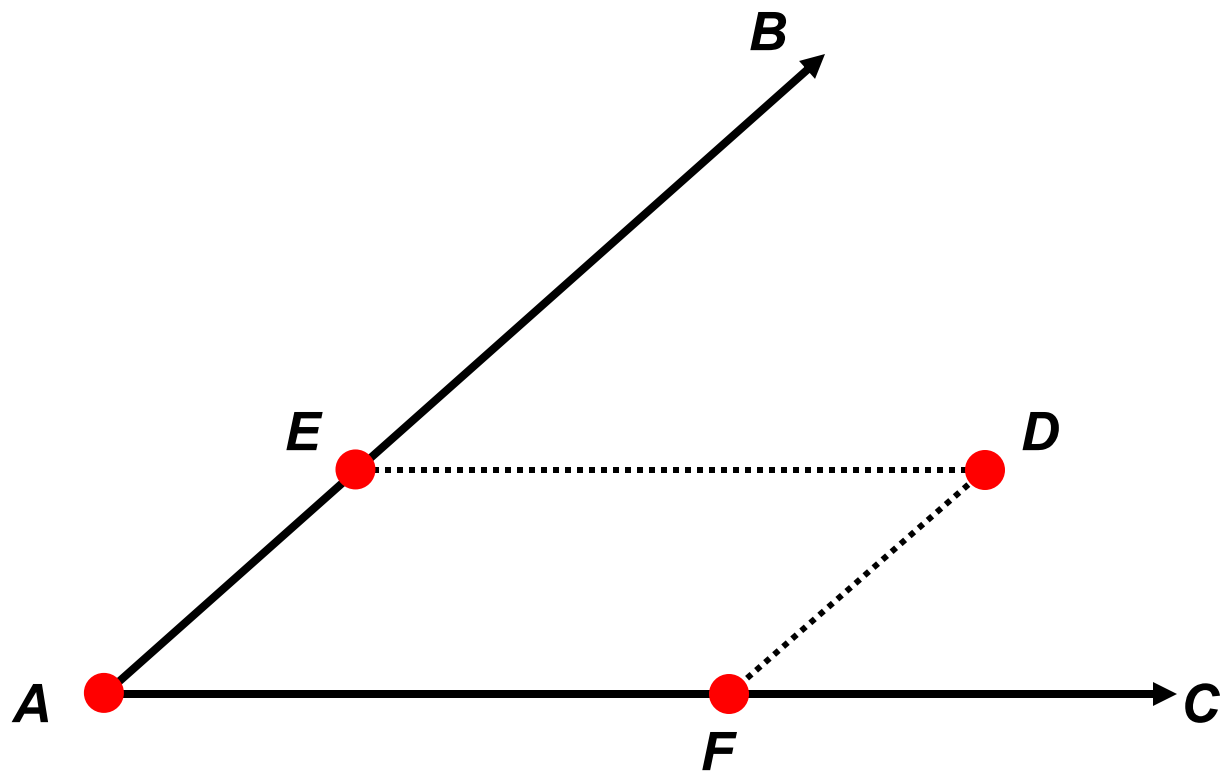
$$X_1 = B_1 + u_1 \times (A_1 - B_1) + v_1 \times (C_1 - B_1)$$

$$X_2 = B_2 + u_2 \times (A_2 - B_2) + v_2 \times (C_2 - B_2)$$

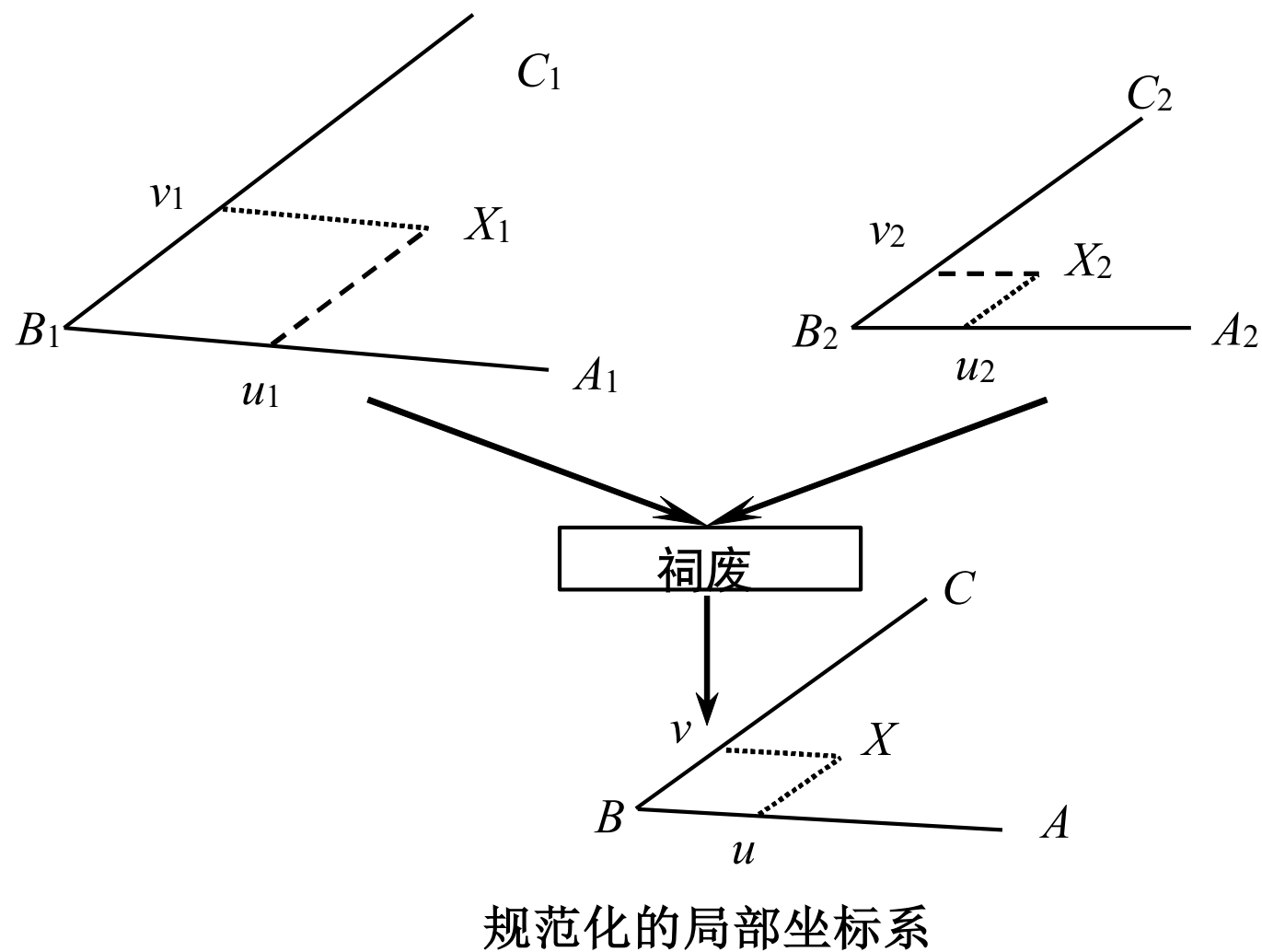
- 给定顶点 $X_1, X_2, (u_1, v_1)$ 和 (u_2, v_2) 很容易根据上述方程组求得。

斜坐标系

- 建坐标系的本质是寻找两个单位向量来表示坐标系内的任意向量。



中间帧局部坐标系的插值



计算中间帧多边形

- 为了计算顶点 X_1 和 X_2 的插值点 X ，取 X 点的规范化局部坐标 (u, v) 为 (u_1, v_1) 和 (u_2, v_2) 的线性插值，即：

$$u = (1-t)u_1 + tu_2, \quad v = (1-t)v_1 + tv_2$$

- 从而得到插值点 X 的位置，

$$X = B + u \times (A - B) + v \times (C - B)$$

- 把上述方法应用到多边形的所有顶点对，我们得到给定多边形的中间帧多边形。这种插值方法可处理包括平移、旋转、比例缩放和剪切变换在内的所有的仿射变换。

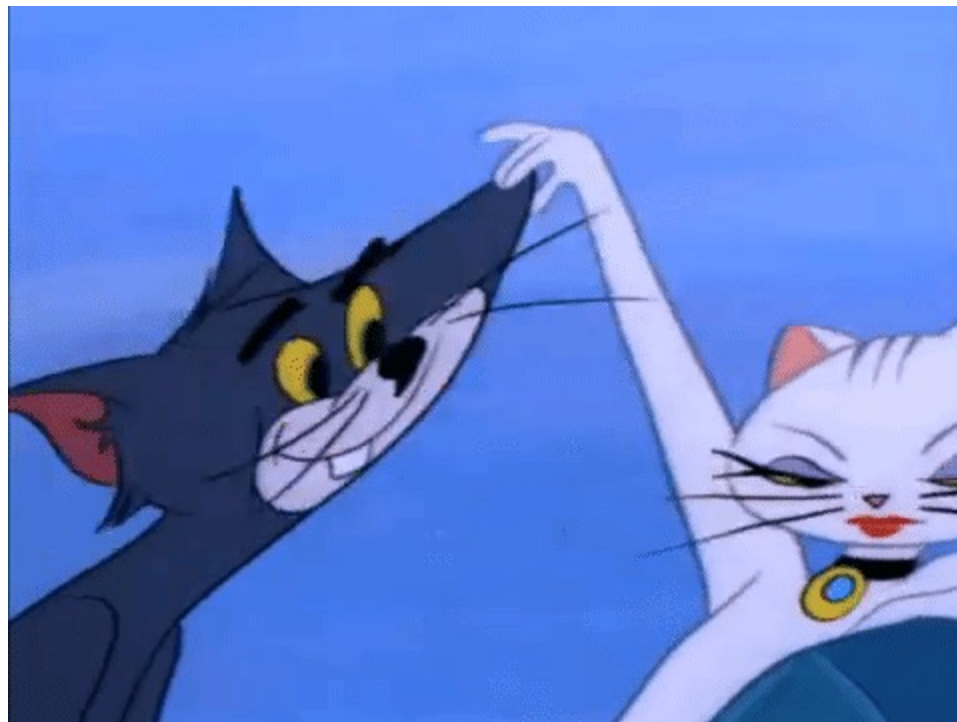
More 2D Shape Blending DEMO



Morphing in After Effects

总结

- 2D Shape Blending 是传统关键帧动画设计中经常碰到的问题，可节约动画师设计中间帧的工作量。
- **局限性：**自动计算得到的结果不一定是动画师所需要的。



总结

- 二维多边形形状渐变
 - 顶点的插值问题
 - 线性插值
 - 基于内在形状的插值：乌龟几何表示+Lagrange乘数法优化
 - 顶点的对应关系以及顶点之间的插值
 - 对应关系：模糊集，隶属函数，模糊相似图，找最长合法完全路径
 - 插值：选取好的仿射变换，插值得到中间帧局部坐标系，规范化局部坐标插值

The End